MATLAB Syntax Notes

Variables, Operators, Loops, Functions and More — Explained

Contents

1. Comments 1
2. Variables and Assignment 2
3. Data Types 2
4. Basic Arithmetic Operators 3
5. Relational Operators 3
6. Logical Operators 3
7. Vectors 4
8. Matrices 4
9. Strings and Char Arrays 5
10. Input and Output 5
11. if / elseif / else 6
12. switch / case 6
13. for Loops 7
14. while Loops 7
15. break and continue 7
16. Functions 8
17. Common Built-in Functions 8

Summary — Quick Reference Cheat Sheet 9

1. Comments

```
% This is a single-line comment

%{
This is a
block comment
%}
```

Anything after a `%` on a line is ignored by MATLAB. Use comments to explain what your code does — it costs nothing and saves you (and your group members) a lot of confusion later. A block comment, wrapped in `%{ ... %}`, lets you comment out several lines at once; the `%{` and `%}` must each sit alone on their own line.

---

2. Variables and Assignment

```
x = 5;
y = 3.2;
name = 'Kwame';
```

A variable is created the moment you assign a value to it — there's no need to declare a type first. `x = 5;` stores the number 5 in a variable called `x`. The semicolon `;` at the end **suppresses output** — without it, MATLAB prints the value of `x` immediately in the Command Window. This is one of the most common early mistakes: forgetting the semicolon and getting your screen flooded with output.

Useful variable-related commands:

```
who         % lists variable names currently in memory
whos        % lists variables with size, type, and memory info
clear x     % deletes variable x from memory
clear all   % deletes all variables from memory
clc         % clears the Command Window (does not delete variables)
```

`who` and `whos` are your quick way to check what you've already defined without scrolling back up. `clear` is useful when you want to make sure a script starts with a completely clean workspace (many students put `clear; clc;` at the very top of a script for this reason).

---

3. Data Types

```
a = 10;              % double (default numeric type)
b = int32(10);       % 32-bit integer
c = 'H';              % char (single character)
d = "Hello";          % string (double-quoted)
e = true;              % logical
```

By default, every number in MATLAB is stored as a `double` (double-precision floating point), even if it looks like a whole number. You rarely need to worry about this unless you're doing something specialised like image processing, where types like `int32` or `uint8` matter.

`'single quotes'` create a **char array** (a row of individual characters), while `"double quotes"` create a **string** (a newer, more convenient text type introduced in MATLAB R2016b). For class work either works, but don't mix them carelessly — `'abc' == "abc"` does not always behave the way you'd expect.

`true` and `false` are **logical** values — MATLAB's version of Boolean. They print as `1` and `0` but are treated specially in conditions.

To check a variable's type: `class(x)`. To check its size: `size(x)`.

---

4. Basic Arithmetic Operators

```
+   addition
-   subtraction
*   multiplication
/   division
^   power (exponent)
```

These behave as expected for scalars: `3 + 4` gives 7, `2^3` gives 8. The catch comes when you use `*`, `/`, or `^` on **matrices** — MATLAB will try to do proper matrix multiplication/division/power, which has strict size rules. For example, `A * B` only works if the number of columns in `A` matches the number of rows in `B`.

If you want the operation applied **element-by-element** instead (i.e., multiply corresponding entries), put a dot in front:

```
.*   element-wise multiplication
./   element-wise division
.^   element-wise power
```

This distinction (`*` vs `.*`) trips up almost everyone at first, so it's worth remembering deliberately: no dot = matrix rules, dot = "do this to every element separately."

---

5. Relational Operators

```
==   equal to
~=   not equal to
<    less than
>    greater than
<=   less than or equal to
>=   greater than or equal to
```

These compare two values and return a logical result (`1` for true, `0` for false). Note that MATLAB uses `~=` for "not equal", not `!=` like some other languages. Also note `==` (double equals) is comparison, while `=` (single equals) is assignment — mixing these up is a classic bug (e.g. writing `if x = 5` instead of `if x == 5` will actually throw an error in MATLAB, which is a small mercy).

When applied to vectors or matrices, relational operators compare element-by-element and return a logical array of the same size, e.g. `[1 2 3] > 2` returns `[0 0 1]`.

---

6. Logical Operators

```
&&   logical AND (short-circuit, for scalars)
||   logical OR  (short-circuit, for scalars)
&    element-wise AND
```

```
|    element-wise OR
~    logical NOT
```

&& and || are used when you're combining two single true/false conditions, typically inside an `if` statement — e.g. `if x > 0 && x < 10`. They are "short-circuit," meaning if the first condition already decides the answer, MATLAB doesn't bother evaluating the second one.

& and | do the same job but element-by-element on arrays, so they're used when comparing vectors or matrices, e.g. `(v > 0) & (v < 10)`.

~ flips a logical value: `~true` gives `false`.

---

7. Vectors

```
v = [1 2 3 4 5];          % row vector
w = [1; 2; 3; 4; 5];      % column vector
r = 1:5;                   % same as [1 2 3 4 5]
r2 = 1:2:10;               % start:step:end -> [1 3 5 7 9]
lin = linspace(0, 1, 5); % 5 evenly spaced points from 0 to 1
```

A vector is just a list of numbers. Square brackets `[...]` build one directly, with **spaces or commas** separating entries in a row, and **semicolons** separating rows (which is how you make a column vector instead).

The **colon operator** `start:end` is a fast way to generate a sequence, and `start:step:end` lets you control the spacing (including negative steps, e.g. `10:-1:1` counts down). `linspace(a, b, n)` is the alternative when you know exactly how many points you want rather than the step size — it always includes both endpoints.

Indexing a vector:

```
v(1)      % first element (MATLAB indexing starts at 1, not 0!)
v(end)    % last element
v(2:4)    % elements 2 through 4
```

This is another classic trap for students coming from languages like Python or C: **MATLAB indexing starts at 1**, not 0. `v(0)` will throw an error.

---

8. Matrices

```
A = [1 2 3; 4 5 6; 7 8 9];   % 3x3 matrix
Z = zeros(3,3);               % 3x3 matrix of zeros
O = ones(2,4);                % 2x4 matrix of ones
I = eye(3);                   % 3x3 identity matrix
R = rand(2,2);                % 2x2 matrix of random numbers (0 to 1)
```

A matrix is built the same way as a vector, just with semicolons marking the end of each row. `zeros`, `ones`, `eye`, and `rand` are quick ways to generate common starting

matrices instead of typing every entry by hand — very handy when you need to pre-allocate space before filling a matrix in a loop.

Indexing a matrix:

```
A(2,3)      % element in row 2, column 3
A(2,:)      % entire row 2
A(:,1)      % entire column 1
A(:)        % all elements, reshaped into a single column
```

The colon `:` on its own inside an index means "everything along this dimension." So `A(2,:)` reads as "row 2, all columns."

Useful matrix functions:

```
size(A)       % returns [rows, columns]
length(A)     % returns the largest dimension
numel(A)      % returns total number of elements
A'            % transpose of A (rows become columns)
```

---

9. Strings and Char Arrays

```
s1 = 'hello';
s2 = "hello";
s3 = strcat(s1, ' world');      % concatenation (char)
s4 = s2 + " world";               % concatenation (string, uses +)
n = num2str(42);                 % number -> string/char
x = str2num('42');               % string -> number
```

`strcat` joins char arrays together. If you're using the newer `"string"` type instead, you can join them with a plain +, which many students find more intuitive. `num2str` and `str2double`/`str2num` convert between numbers and text — useful when you want to build a message that includes a calculated value, since you can't directly glue a number onto text with + unless it's already text.

---

10. Input and Output

```
disp('Hello World');            % display text or a variable
disp(x);
fprintf('Value is %d\n', x);     % formatted output
fprintf('Pi is %.2f\n', pi);      % 2 decimal places
name = input('Enter your name: ', 's');  % get text input from user
num = input('Enter a number: ');           % get numeric input
```

`disp` is the simple option — it just prints whatever you give it, with no extra formatting. `fprintf` gives you control over formatting using **format specifiers**: `%d` for integers, `%f` for decimals (with `%.2f` limiting it to 2 decimal places), `%s` for strings, and `\n` to move to a new line.

`input` pauses the program and waits for the user to type something. By default it expects a number; adding `'s'` as the second argument tells it to treat the input as text instead.

---

11. if / elseif / else

```
x = 7;

if x > 10
    disp('Greater than 10');
elseif x > 5
    disp('Greater than 5 but not more than 10');
else
    disp('5 or less');
end
```

MATLAB checks conditions from top to bottom and runs the **first** block whose condition is true, then skips the rest. `elseif` (written as one word, unlike some languages) lets you chain multiple conditions, and `else` catches anything not covered above. Every `if` block must be closed with end — forgetting this is one of the most common syntax errors for beginners.

---

12. switch / case

```
grade = 'B';

switch grade
    case 'A'
        disp('Excellent');
    case 'B'
        disp('Good');
    case 'C'
        disp('Average');
    otherwise
        disp('Needs improvement');
end
```

`switch` is a cleaner alternative to a long chain of `elseif` statements when you're comparing one variable against several fixed possible values. It checks `grade` against each `case` in order and runs the matching block; `otherwise` acts like a final `else`. Like `if`, it must be closed with end.

---

13. for Loops

```
for i = 1:5
    disp(i);
end
```

A `for` loop repeats a block of code once for each value in a given range — here, `i` takes the values 1, 2, 3, 4, 5 in turn, printing each one. The loop variable (`i` here) doesn't need to be pre-declared, and it keeps the last value it had once the loop finishes.

You can loop over any vector, not just a simple range:

```
scores = [55 70 90];
for s = scores
    disp(s);
end
```

`for` loops are the natural choice when you know in advance how many times you need to repeat something (e.g. "do this for every row of a matrix" or "do this 10 times").

---

14. while Loops

```
n = 1;
while n <= 5
    disp(n);
    n = n + 1;
end
```

A `while` loop repeats a block of code **as long as** its condition stays true. Unlike a `for` loop, you don't know in advance how many times it will run — it depends entirely on the condition. It's your responsibility to make sure something inside the loop eventually makes the condition false (here, incrementing `n`), or you'll get an **infinite loop** that never stops (Ctrl+C in the Command Window will forcibly stop it).

`while` loops suit situations where you're repeating something "until a certain state is reached" rather than "a fixed number of times" — e.g. waiting until a value converges, or until a user enters valid input.

---

15. break and continue

```
for i = 1:10
    if i == 5
        break;      % exits the loop completely
    end
    disp(i);
end

for i = 1:5
```

```
    if mod(i,2) == 0
        continue;   % skips the rest of this iteration, moves to next
    end
    disp(i);
end
```

`break` immediately exits the loop it's inside, no matter how many iterations were left. `continue` doesn't exit the loop — it just skips whatever code comes after it **for that one iteration** and jumps straight to the next one. Both work inside `for` and `while` loops.

---

16. Functions

```
function result = addNumbers(a, b)
    result = a + b;
end
```

A function packages a piece of logic so you can reuse it without retyping it. `result = addNumbers(a, b)` defines a function called `addNumbers` that takes two inputs (`a`, `b`) and returns one output (`result`). When saved in its own file, the filename must match the function name exactly (`addNumbers.m`), which is a MATLAB-specific rule that catches a lot of people out.

You can call it afterwards just like any built-in function:

```
total = addNumbers(3, 4);   % total = 7
```

Anonymous (inline) functions are a shorter way to define small, throwaway functions without a separate file:

```
square = @(x) x^2;
square(5)     % returns 25
```

The `@(x)` part lists the input(s); everything after it is the expression that gets evaluated. These are especially handy when a MATLAB function (like a plotting or optimisation function) expects you to pass in another function as an argument.

---

17. Common Built-in Functions

```
sum(v)        % adds up all elements
mean(v)       % average of elements
max(v)        % largest value
min(v)        % smallest value
sort(v)       % sorts elements in ascending order
find(v > 3)   % returns indices where condition is true
abs(x)        % absolute value
sqrt(x)       % square root
```

```
round(x)      % rounds to nearest integer
mod(a,b)      % remainder of a divided by b
```

These are ready-made functions that cover most everyday tasks so you don't need to write loops for them yourself. `find` is particularly useful — it doesn't return the values that meet a condition, it returns their **positions** (indices) in the array, which you can then use to look up or modify those entries.

Summary — Quick Reference Cheat Sheet

Category	Syntax	Meaning
Comment	`%`, `%{ ... %}`	Single-line / block comment
Assignment	`x = 5;`	Creates variable; `;` suppresses output
Workspace	`who`, `whos`, `clear`, `clc`	Inspect / clear variables and screen
Data types	`double`, `int32`, `char`, `string`, `logical`	Number, integer, text, string, true/false
Arithmetic	`+ - * / ^`	Standard math (matrix rules apply)
Element-wise	`.* ./ .^`	Apply operation entry-by-entry
Relational	`== ~= < > <= >=`	Comparison, returns logical
Logical (scalar)	`&& ||`	AND / OR, short-circuit
Logical (array)	`& | ~`	Element-wise AND / OR / NOT
Vector creation	`[1 2 3]`, `a:b`, `a:step:b`, `linspace(a,b,n)`	Build a list of numbers
Matrix creation	`[1 2;3 4]`, `zeros`, `ones`, `eye`, `rand`	Build a 2D array
Indexing	`v(i)`, `A(r,c)`, `A(:,c)`, `A(r,:)`, `end`	Access elements (1-based!)
Matrix info	`size`, `length`, `numel`, `'` (transpose)	Dimensions and shape
Strings	`'text'`, `"text"`, `strcat`, `num2str`, `str2num`	Text handling and conversion
Output	`disp(x)`, `fprintf('%d\n', x)`	Print to Command Window
Input	`input('prompt')`, `input('prompt','s')`	Get number / text from user

Category	Syntax	Meaning
Conditional	`if / elseif / else / end`	Branch based on condition
Multi-way branch	`switch / case / otherwise / end`	Compare one variable to several values
Counted loop	`for i = a:b ... end`	Repeat a known number of times
Conditional loop	`while cond ... end`	Repeat until condition is false
Loop control	`break`, `continue`	Exit loop / skip to next iteration
Function (file)	`function out = name(in) ... end`	Reusable named block of code
Function (inline)	`f = @(x) expression;`	Quick one-line function
Common built-ins	`sum, mean, max, min, sort, find, abs, sqrt, round, mod`	Everyday array/math operations

Big things to remember as beginners:

1. MATLAB indexing starts at **1**, not 0.
2. `;` suppresses output — leave it off only when you want to see the result.
3. *, /, ^ follow matrix rules; add a `.` (`.*`, `./`, `.^`) for element-wise operations.
4. Every `if`, `for`, `while`, `switch`, and `function` block must be closed with `end`.
5. A function saved in its own file must have a filename that matches the function name exactly.